Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота 5

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

«**Розробка багатовіконного інтерфейсу користувача для графічного редактора об’єктів** »

Виконав:

студент групи ІМ-55

Бабенко Ярослав Володимирович

Номер в списку - 3

Перевірив:

Рекечинський Д.О.

Київ 2026

Мета роботи

Отримати вміння та навички програмувати багатовіконний інтерфейс програми в об’єктно-орієнтованому стилі.

1. Завдання

1. Створити у середовищі розробки проект з ім'ям Lab5.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

2. Варіант завдання та основні вимоги

Розрахунковий номер варіанту дорівнює номеру в журналі: **$Ж = 3$** (непарний номер).

1. **Архітектура MyEditor:** Оскільки номер варіанту непарний ($1, 3, 5\dots$), запрограмовано глобальний статичний об'єкт класу `MyEditor` у вигляді **Singleton Меєрса**.
2. **Стилі та фігури:** Усі кольори та типи геометричних форм повністю успадковані з попередньої лабораторної роботи №4 (6 фігур, пунктирний слід).
3. **Вікно таблиці:**
    a. Запрограмовано як **немодальне вікно діалогу**.
    b. Винесено в окремий незалежний модуль (клас `MyTable`).
    c. Відкривається/закривається через окремий пункт меню "Об'єкти -> Таблиця фігур".
    d. Автоматично закривається при виході з основної програми.
    e. Містить таблицю `ListView` з колонками: *Назва, x1, y1, x2, y2*.
4. **Робота з файлами:** Запрограмовано збереження множини введених об'єктів у текстовий файл формату **CSV** із п'ятьма полями (тип об'єкту та 4 координати).
5. **Бонуси-заохочення (Реалізовано):**
    a. При виділенні курсором рядка в таблиці відповідний об'єкт підсвічується на головному зображенні (Бонус 1).
    b. При подвійному кліку на рядок таблиці відповідний об'єкт вилучається з масиву об'єктів (Бонус 2).
    c. Для ізоляції модуля таблиці від інших `.xxx` файлів використано надсилання кастомних повідомлень (`WM_USER`), що реалізує патерн **Observer/Callback**.

Код вихідних файлів:

editor.h

```
#pragma once

#include <windows.h>
#include <commctrl.h>
#include <string>
#include <fstream>
#include "shape.h"

#define MAX_SHAPES 103

#define ID_OBJECT_POINT    32771
#define ID_OBJECT_LINE     32772
#define ID_OBJECT_RECT     32773
#define ID_OBJECT_ELLIPSE  32774
#define ID_OBJECT_LINEOO   32775
#define ID_OBJECT_CUBE     32776

class MyEditor {
private:
   HWND hWnd;
   HWND hToolBar;
   Shape** pcshape;
   int shapeCount;
   int currentTool;
   bool isDrawing;
   long startX, startY, endX, endY;
   Shape* tempShape;
   HPEN hRubberPen;

   MyEditor();
   ~MyEditor();
   MyEditor(const MyEditor&) = delete;
   MyEditor& operator=(const MyEditor&) = delete;

   int selectedShapeIndex = -1;

   Shape* CreateShapeFactory(int tool);

public:
   static MyEditor& getInstance() {
      static MyEditor instance;
      return instance;
   }
```

```
    void Start(HWND h);
    void CreateToolbar();
    void OnInitMenuPopup(WPARAM wParam, LPARAM lParam);
    LRESULT OnNotify(LPARAM lParam);

    void OnLButtonDown(LPARAM lParam);
    void OnMouseMove(LPARAM lParam);
    void OnLButtonUp(LPARAM lParam);
    void OnPaint();
    void OnCommand(WPARAM wParam);

    bool SaveToFile(const std::wstring& filename);

    void ClearAllShapes();

    bool LoadFromFile(const std::wstring& filename);

    void HighlightShape(int index);
    void DeleteShape(int index);

    int GetShapeCount() const { return shapeCount; }
    Shape* GetShape(int index) const {
        if (index >= 0 && index < shapeCount) return pcshape[index];
        return nullptr;
    }
};
```

editor.cpp #include "editor.h" #include // [ДОДАТКОВЕ ЗАВДАННЯ 3]: Для роботи з рядковими потоками std::wstringstream

MyEditor::MyEditor() : hWnd(NULL), hToolBar(NULL), shapeCount(0), currentTool(0), isDrawing(false), startX(0), startY(0), endX(0), endY(0), tempShape(nullptr), hRubberPen(nullptr) { pcshape = new Shape * [MAX_SHAPES]; for (int i = 0; i < MAX_SHAPES; i++) { pcshape[i] = nullptr; } }

MyEditor::~MyEditor() { ClearAllShapes(); // Використовуємо уніфіковану функцію очищення if (pcshape) { delete[] pcshape; pcshape = nullptr; } if (tempShape) { delete tempShape; tempShape = nullptr; } if (hRubberPen) { DeleteObject(hRubberPen); hRubberPen = nullptr; } }

void MyEditor::Start(HWND h) { hWnd = h; CreateToolbar(); }

Shape* MyEditor::CreateShapeFactory(int tool) { switch (tool) { case 0: return new PointShape(); case 1: return new LineShape(); case 2: return new RectShape(); case 3: return new EllipseShape(); case 4: return new LineOOShape(); case 5: return new CubeShape(); } return nullptr; }

```
void MyEditor::CreateToolbar() { InitCommonControls(); HIMAGELIST hImageList =
ImageList_Create(16, 16, ILC_COLOR24 | ILC_MASK, 6, 0);

HDC hdcScreen = GetDC(hWnd);
HDC hdcMem = CreateCompatibleDC(hdcScreen);
HBITMAP hBmp = CreateCompatibleBitmap(hdcScreen, 96, 16);
HBITMAP hOldBmp = (HBITMAP)SelectObject(hdcMem, hBmp);

COLORREF maskColor = RGB(255, 0, 255);
HBRUSH hBgBrush = CreateSolidBrush(maskColor);
RECT rcFull = { 0, 0, 96, 16 };
FillRect(hdcMem, &rcFull, hBgBrush);
DeleteObject(hBgBrush);

HPEN hBlackPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
HPEN hOldPen = (HPEN)SelectObject(hdcMem, hBlackPen);

// 0: Точка
HBRUSH hBlackBrush = CreateSolidBrush(RGB(0, 0, 0));
HBRUSH hOldBrush = (HBRUSH)SelectObject(hdcMem, hBlackBrush);
Ellipse(hdcMem, 6, 6, 11, 11);
SelectObject(hdcMem, hOldBrush); DeleteObject(hBlackBrush);

// 1: Лінія
MoveToEx(hdcMem, 16 + 3, 12, NULL); LineTo(hdcMem, 16 + 13, 3);

// 2: Прямокутник
hOldBrush = (HBRUSH)SelectObject(hdcMem,
GetStockObject(NULL_BRUSH));
Rectangle(hdcMem, 32 + 3, 3, 32 + 13, 13);
SelectObject(hdcMem, hOldBrush);

// 3: Еліпс
HBRUSH hPinkBrush = CreateSolidBrush(RGB(255, 192, 203));
hOldBrush = (HBRUSH)SelectObject(hdcMem, hPinkBrush);
Ellipse(hdcMem, 48 + 2, 4, 48 + 14, 12);
SelectObject(hdcMem, hOldBrush); DeleteObject(hPinkBrush);

// 4: Лінія з кружечками
MoveToEx(hdcMem, 64 + 4, 11, NULL); LineTo(hdcMem, 64 + 12, 4);
hOldBrush = (HBRUSH)SelectObject(hdcMem,
GetStockObject(NULL_BRUSH));
```

```
Ellipse(hdcMem, 64 + 2, 9, 64 + 6, 13);
Ellipse(hdcMem, 64 + 10, 2, 64 + 14, 6);
SelectObject(hdcMem, hOldBrush);

// 5: Ky6
hOldBrush = (HBRUSH)SelectObject(hdcMem,
GetStockObject(NULL_BRUSH));
Rectangle(hdcMem, 80 + 2, 6, 80 + 11, 14);
Rectangle(hdcMem, 80 + 5, 2, 80 + 14, 10);
MoveToEx(hdcMem, 80 + 2, 6, NULL);   LineTo(hdcMem, 80 + 5, 2);
MoveToEx(hdcMem, 80 + 11, 6, NULL);  LineTo(hdcMem, 80 + 14, 2);
MoveToEx(hdcMem, 80 + 2, 14, NULL);  LineTo(hdcMem, 80 + 5, 10);
MoveToEx(hdcMem, 80 + 11, 14, NULL); LineTo(hdcMem, 80 + 14, 10);
SelectObject(hdcMem, hOldBrush);

SelectObject(hdcMem, hOldPen); DeleteObject(hBlackPen);
SelectObject(hdcMem, hOldBmp); DeleteDC(hdcMem); ReleaseDC(hWnd,
hdcScreen);

ImageList_AddMasked(hImageList, hBmp, maskColor);
DeleteObject(hBmp);

TBBUTTON tbb[6]; ZeroMemory(tbb, sizeof(tbb));
tbb[0].iBitmap = 0; tbb[0].idCommand = ID_OBJECT_POINT;
tbb[0].fsState = TBSTATE_ENABLED | TBSTATE_CHECKED; tbb[0].fsStyle =
TBSTYLE_CHECKGROUP;
tbb[1].iBitmap = 1; tbb[1].idCommand = ID_OBJECT_LINE;
tbb[1].fsState = TBSTATE_ENABLED;                   tbb[1].fsStyle =
TBSTYLE_CHECKGROUP;
tbb[2].iBitmap = 2; tbb[2].idCommand = ID_OBJECT_RECT;
tbb[2].fsState = TBSTATE_ENABLED;                   tbb[2].fsStyle =
TBSTYLE_CHECKGROUP;
tbb[3].iBitmap = 3; tbb[3].idCommand = ID_OBJECT_ELLIPSE;
tbb[3].fsState = TBSTATE_ENABLED;                   tbb[3].fsStyle =
TBSTYLE_CHECKGROUP;
tbb[4].iBitmap = 4; tbb[4].idCommand = ID_OBJECT_LINEOO;
tbb[4].fsState = TBSTATE_ENABLED;                   tbb[4].fsStyle =
TBSTYLE_CHECKGROUP;
tbb[5].iBitmap = 5; tbb[5].idCommand = ID_OBJECT_CUBE;
tbb[5].fsState = TBSTATE_ENABLED;                   tbb[5].fsStyle =
TBSTYLE_CHECKGROUP;
```

```
hToolBar = CreateWindowEx(0, TOOLBARCLASSNAME, NULL, WS_CHILD |
WS_VISIBLE | WS_BORDER | TBSTYLE_TOOLTIPS,
    0, 0, 0, 0, hWnd, (HMENU)1001, GetModuleHandle(NULL), NULL);
SendMessage(hToolBar, TB_BUTTONSTRUCTSIZE, (WPARAM)sizeof(TBBUTTON),
0);
SendMessage(hToolBar, TB_SETIMAGELIST, 0, (LPARAM)hImageList);
SendMessage(hToolBar, TB_ADDBUTTONS, (WPARAM)6, (LPARAM)&tbb);
SendMessage(hToolBar, TB_AUTOSIZE, 0, 0);


}
```

void MyEditor::OnLButtonDown(LPARAM lParam) { if (shapeCount < MAX_SHAPES) { isDrawing = true; startX = endX = LOWORD(lParam); startY = endY = HIWORD(lParam); tempShape = CreateShapeFactory(currentTool); hRubberPen = CreatePen(PS_DOT, 1, RGB(0, 0, 0)); }}

void MyEditor::OnMouseMove(LPARAM lParam) { if (isDrawing && tempShape) { HDC hdc = GetDC(hWnd); SetROP2(hdc, R2_NOTXORPEN); SetBkMode(hdc, TRANSPARENT); HPEN oldPen = (HPEN)SelectObject(hdc, hRubberPen);

```
   tempShape->Set(startX, startY, endX, endY);
    tempShape->Show(hdc);
    endX = LOWORD(lParam);
    endY = HIWORD(lParam);
    tempShape->Set(startX, startY, endX, endY);
    tempShape->Show(hdc);

    SelectObject(hdc, oldPen);
    ReleaseDC(hWnd, hdc);
}


}
```

void MyEditor::OnLButtonUp(LPARAM lParam) { if (isDrawing) { isDrawing = false; pcshape[shapeCount] = CreateShapeFactory(currentTool); pcshape[shapeCount]->Set(startX, startY, endX, endY); shapeCount++;

```
   delete tempShape; tempShape = nullptr;
    if (hRubberPen) { DeleteObject(hRubberPen); hRubberPen =
nullptr; }

    selectedShapeIndex = -1;
```

```
    InvalidateRect(hWnd, NULL, TRUE);
}


}
```

void MyEditor::OnPaint() { PAINTSTRUCT ps; HDC hdc = BeginPaint(hWnd, &ps);

```
for (int i = 0; i < shapeCount; i++) {
    if (pcshape[i]) {
        if (i == selectedShapeIndex) {
            HPEN hHighlightPen = CreatePen(PS_SOLID, 3, RGB(255, 0,
0));
            HPEN hOldPen = (HPEN)SelectObject(hdc, hHighlightPen);
            int oldBkMode = SetBkMode(hdc, TRANSPARENT);

            pcshape[i]->Show(hdc);

            SetBkMode(hdc, oldBkMode);
            SelectObject(hdc, hOldPen);
            DeleteObject(hHighlightPen);
        }
        else {
            pcshape[i]->Show(hdc);
        }
    }
}
EndPaint(hWnd, &ps);


}
```

bool MyEditor::SaveToFile(const std::wstring& filename) { std::ofstream file(filename, std::ios::out | std::ios::trunc | std::ios::binary); if (!file.is_open()) return false;

```
auto toUtf8 = [](const std::wstring& wstr) -> std::string {
    if (wstr.empty()) return "";
    int sizeNeeded = WideCharToMultiByte(CP_UTF8, 0, wstr.c_str(),
(int)wstr.size(), NULL, 0, NULL, NULL);
    std::string strTo(sizeNeeded, 0);
    WideCharToMultiByte(CP_UTF8, 0, wstr.c_str(), (int)wstr.size(),
&strTo[0], sizeNeeded, NULL, NULL);
    return strTo;
    };
```

```
unsigned char bom[] = { 0xEF, 0xBB, 0xBF };
file.write((char*)bom, sizeof(bom));

static const WCHAR csvHeader[] = { 0x041D, 0x0430, 0x0437, 0x0432,
0x0430, ';', 'x', '1', ';', 'y', '1', ';', 'x', '2', ';', 'y', '2',
'\n', 0 };
file << toUtf8(csvHeader);

for (int i = 0; i < shapeCount; i++) {
    if (pcshape[i]) {
        std::wstring rowW = pcshape[i]->ToCSV() + L"\n";
        file << toUtf8(rowW);
    }
}

file.flush();
file.close();
return true;


}
```

void MyEditor::ClearAllShapes() { for (int i = 0; i < shapeCount; i++) { if (pcshape[i]) { delete pcshape[i]; pcshape[i] = nullptr; } } shapeCount = 0; selectedShapeIndex = -1; if (hWnd) InvalidateRect(hWnd, NULL, TRUE); }

bool MyEditor::LoadFromFile(const std::wstring& filename) { std::ifstream file(filename, std::ios::binary); if (!file.is_open()) return false;

```
file.seekg(0, std::ios::end);
std::streampos fileSize = file.tellg();
if (fileSize <= 0) {
    file.close();
    return false;
}
file.seekg(0, std::ios::beg);

unsigned char bom[3] = { 0 };
file.read((char*)bom, 3);
if (!(bom[0] == 0xEF && bom[1] == 0xBB && bom[2] == 0xBF)) {
    file.seekg(0, std::ios::beg);
}
```

```
ClearAllShapes();

auto utf8ToWstring = [](const std::string& str) -> std::wstring {
    if (str.empty()) return L"";
    int sizeNeeded = MultiByteToWideChar(CP_UTF8, 0, str.c_str(),
(int)str.size(), NULL, 0);
    if (sizeNeeded <= 0) {
        sizeNeeded = MultiByteToWideChar(CP_ACP, 0, str.c_str(),
(int)str.size(), NULL, 0);
        std::wstring wstr(sizeNeeded, 0);
        MultiByteToWideChar(CP_ACP, 0, str.c_str(), (int)str.size(),
&wstr[0], sizeNeeded);
        return wstr;
    }
    std::wstring wstr(sizeNeeded, 0);
    MultiByteToWideChar(CP_UTF8, 0, str.c_str(), (int)str.size(),
&wstr[0], sizeNeeded);
    return wstr;
    };

std::string lineA;
if (!std::getline(file, lineA)) {
    file.close();
    return false;
}

while (std::getline(file, lineA) && shapeCount < MAX_SHAPES) {
    if (!lineA.empty() && lineA.back() == '\r') lineA.pop_back();
    if (lineA.empty()) continue;

    std::wstring lineW = utf8ToWstring(lineA);
    std::wstringstream ss(lineW);
    std::wstring name, sX1, sY1, sX2, sY2;

    if (std::getline(ss, name, L';') &&
        std::getline(ss, sX1, L';') &&
        std::getline(ss, sY1, L';') &&
        std::getline(ss, sX2, L';') &&
        std::getline(ss, sY2, L';'))
    {
        long x1 = std::stol(sX1);
```

```
        long y1 = std::stol(sY1);
        long x2 = std::stol(sX2);
        long y2 = std::stol(sY2);

        Shape* newShape = nullptr;

        static const WCHAR namePoint[] = { 0x041A, 0x0440, 0x0430,
0x043F, 0x043A, 0x0430, 0 }; // Крапка
        static const WCHAR nameLine[] = { 0x041B, 0x0456, 0x043D,
0x0456, 0x044F, 0 };          // Лінія
        static const WCHAR nameRect[] = { 0x041F, 0x0440, 0x044F,
0x043C, 0x043E, 0x043A, 0x0443, 0x0442, 0x043D, 0x0438, 0x043A, 0 };
// Прямокутник
        static const WCHAR nameEllipse[] = { 0x0415, 0x043B, 0x0456,
0x043F, 0x0441, 0 };       // Еліпс
        static const WCHAR nameLineOO[] = { 0x041B, 0x0456, 0x043D,
0x0456, 0x044F, ' ', 0x0437, ' ', 0x043A, 0x0443, 0x0436, 0x0435,
0x0447, 0x043A, 0x0430, 0x043C, 0x0438, 0 }; // Лінія з кружечками
        static const WCHAR nameCube[] = { 0x041A, 0x0430, 0x0440,
0x043A, 0x0430, 0x0441, ' ', 0x043A, 0x0443, 0x0431, 0x0430, 0 }; //
Каркас куба

        if (name == namePoint)        newShape = new PointShape();
        else if (name == nameLine)    newShape = new LineShape();
        else if (name == nameRect)    newShape = new RectShape();
        else if (name == nameEllipse) newShape = new EllipseShape();
        else if (name == nameLineOO)  newShape = new LineOOShape();
        else if (name == nameCube)    newShape = new CubeShape();

        if (newShape) {
            newShape->Set(x1, y1, x2, y2);
            pcshape[shapeCount++] = newShape;
        }
    }
}

file.close();
if (hWnd) InvalidateRect(hWnd, NULL, TRUE);
return true;


}
```

void MyEditor::HighlightShape(int index) { if (index >= 0 && index < shapeCount) { selectedShapeIndex = index; } else { selectedShapeIndex = -1; } InvalidateRect(hWnd, NULL, TRUE); }

void MyEditor::DeleteShape(int index) { if (index >= 0 && index < shapeCount && pcshape[index] != nullptr) { delete pcshape[index]; pcshape[index] = nullptr;

```
   for (int i = index; i < shapeCount - 1; i++) {
        pcshape[i] = pcshape[i + 1];
    }
    pcshape[shapeCount - 1] = nullptr;
    shapeCount--;
    selectedShapeIndex = -1;
    InvalidateRect(hWnd, NULL, TRUE);
}


}
```

void MyEditor::OnCommand(WPARAM wParam) { int wmId = LOWORD(wParam); switch (wmId) { case ID_OBJECT_POINT: currentTool = 0; break; case ID_OBJECT_LINE: currentTool = 1; break; case ID_OBJECT_RECT: currentTool = 2; break; case ID_OBJECT_ELLIPSE: currentTool = 3; break; case ID_OBJECT_LINEOO: currentTool = 4; break; case ID_OBJECT_CUBE: currentTool = 5; break; } if (hToolBar) SendMessage(hToolBar, TB_CHECKBUTTON, ID_OBJECT_POINT + currentTool, MAKELONG(TRUE, 0)); }

void MyEditor::OnInitMenuPopup(WPARAM wParam, LPARAM lParam) { HMENU hMenu = (HMENU)wParam; for (int i = 0; i < 6; i++) { CheckMenuItem(hMenu, ID_OBJECT_POINT + i, MF_BYCOMMAND | MF_UNCHECKED); } CheckMenuItem(hMenu, ID_OBJECT_POINT + currentTool, MF_BYCOMMAND | MF_CHECKED); }

LRESULT MyEditor::OnNotify(LPARAM lParam) { LPNMHDR pnmh = (LPNMHDR)lParam; if (pnmh->code == TTN_GETDISPINFOW) { LPTOOLTIPTEXTW lpttt = (LPTOOLTIPTEXTW)lParam; lpttt->hinst = NULL; static WCHAR strPoint[] = { 0x041A, 0x0440, 0x0430, 0x043F, 0x043A, 0x0430, 0 }; static WCHAR strLine[] = { 0x041B, 0x0456, 0x043D, 0x0456, 0x044F, 0 }; static WCHAR strRect[] = { 0x041F, 0x0440, 0x044F, 0x043C, 0x043E, 0x043A, 0x0443, 0x0442, 0x043D, 0x0438, 0x043A, 0 }; static WCHAR strEllipse[] = { 0x0415, 0x043B, 0x0456, 0x043F, 0x0441, 0 }; static WCHAR strLineOO[] = { 0x041B, 0x0456, 0x043D, 0x0456, 0x044F, ' ', 0x0437, ' ', 0x043A, 0x0440, 0x0443, 0x0436, 0x0435, 0x0447, 0x043A, 0x0430, 0x043C, 0x0438, 0 }; static WCHAR strCube[] = { 0x041A, 0x0430, 0x0440, 0x043A, 0x0430, 0x0441, ' ', 0x043A, 0x0443, 0x0431, 0x0430, 0 }; switch (lpttt->hdr.idFrom) { case ID_OBJECT_POINT: lpttt->lpszText

= strPoint; break; case ID_OBJECT_LINE: lpttt->lpszText = strLine; break; case ID_OBJECT_RECT: lpttt->lpszText = strRect; break; case ID_OBJECT_ELLIPSE: lpttt->lpszText = strEllipse; break; case ID_OBJECT_LINEOO: lpttt->lpszText = strLineOO; break; case ID_OBJECT_CUBE: lpttt->lpszText = strCube; break; } } return 0; }

shape.h

```
#pragma once

#include <windows.h>
#include <string>

class Shape {
protected:
   long xs1, ys1, xs2, ys2;

public:
   virtual ~Shape() {}

   void Set(long x1, long y1, long x2, long y2);
   virtual void Show(HDC hdc) = 0;



   virtual std::wstring GetName() const = 0;

   long GetX1() const { return xs1; }
   long GetY1() const { return ys1; }
   long GetX2() const { return xs2; }
   long GetY2() const { return ys2; }

   virtual std::wstring ToCSV() const;
};



class PointShape : virtual public Shape {
public:
   void Show(HDC hdc) override;
   std::wstring GetName() const override;
};

class LineShape : virtual public Shape {
public:
   void Show(HDC hdc) override;
   std::wstring GetName() const override;
};
```

```
class RectShape : virtual public Shape {
public:
   void Show(HDC hdc) override;
   std::wstring GetName() const override;
};

class EllipseShape : virtual public Shape {
public:
   void Show(HDC hdc) override;
   std::wstring GetName() const override;
};

class LineOOShape : public LineShape, public EllipseShape {
public:
   void Show(HDC hdc) override;
   std::wstring GetName() const override;
};

class CubeShape : public RectShape, public LineShape {
public:
   void Show(HDC hdc) override;
   std::wstring GetName() const override;
};
```

shape.cpp

```
#include "shape.h"
#include <cmath>



void Shape::Set(long x1, long y1, long x2, long y2) {
   xs1 = x1; ys1 = y1; xs2 = x2; ys2 = y2;
}



std::wstring Shape::ToCSV() const {
   return GetName() + L";" +
      std::to_wstring(xs1) + L";" + std::to_wstring(ys1) + L";" +
      std::to_wstring(xs2) + L";" + std::to_wstring(ys2);
}



void PointShape::Show(HDC hdc) {
   HBRUSH hBrush = CreateSolidBrush(RGB(0, 0, 0));
   HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);
   HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
   HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
   Ellipse(hdc, xs2 - 2, ys2 - 2, xs2 + 2, ys2 + 2);
```

```
    SelectObject(hdc, hOldPen); SelectObject(hdc, hOldBrush);
    DeleteObject(hPen); DeleteObject(hBrush);
}
std::wstring PointShape::GetName() const {
    static const WCHAR name[] = { 0x041A, 0x0440, 0x0430, 0x043F, 0x043A, 0x0430, 0 }; // Крапка
    return name;
}
void LineShape::Show(HDC hdc) {
    MoveToEx(hdc, xs1, ys1, NULL);
    LineTo(hdc, xs2, ys2);
}
std::wstring LineShape::GetName() const {
    static const WCHAR name[] = { 0x041B, 0x0456, 0x043D, 0x0456, 0x044F, 0 }; // Лінія
    return name;
}

void RectShape::Show(HDC hdc) {
    HBRUSH oldBrush = (HBRUSH)SelectObject(hdc, GetStockObject(NULL_BRUSH));
    long rx = std::abs(xs2 - xs1);
    long ry = std::abs(ys2 - ys1);
    Rectangle(hdc, xs1 - rx, ys1 - ry, xs1 + rx, ys1 + ry);
    SelectObject(hdc, oldBrush);
}
std::wstring RectShape::GetName() const {
    static const WCHAR name[] = { 0x041F, 0x0440, 0x044F, 0x043C, 0x043E, 0x043A, 0x0443, 0x0442, 0x043D,
0x0438, 0x043A, 0 }; // Прямокутник
    return name;
}

void EllipseShape::Show(HDC hdc) {
    HBRUSH currentBrush = NULL;
    HBRUSH oldBrush = NULL;
    if (GetROP2(hdc) == R2_NOTXORPEN) {
        currentBrush = (HBRUSH)GetStockObject(NULL_BRUSH);
    }
    else {
        currentBrush = CreateSolidBrush(RGB(255, 192, 203));
    }
    oldBrush = (HBRUSH)SelectObject(hdc, currentBrush);
    Ellipse(hdc, xs1, ys1, xs2, ys2);
    SelectObject(hdc, oldBrush);
    if (GetROP2(hdc) != R2_NOTXORPEN) DeleteObject(currentBrush);
}
std::wstring EllipseShape::GetName() const {
    static const WCHAR name[] = { 0x0415, 0x043B, 0x0456, 0x043F, 0x0441, 0 }; // Еліпс
    return name;
}



void LineOOShape::Show(HDC hdc) {
    long origX1 = xs1, origY1 = ys1;
    long origX2 = xs2, origY2 = ys2;


    LineShape::Set(origX1, origY1, origX2, origY2);
    LineShape::Show(hdc);
```

```
    const long R = 5;
    EllipseShape::Set(origX1 - R, origY1 - R, origX1 + R, origY1 + R);
    EllipseShape::Show(hdc);

    EllipseShape::Set(origX2 - R, origY2 - R, origX2 + R, origY2 + R);
    EllipseShape::Show(hdc);

    Shape::Set(origX1, origY1, origX2, origY2);
}
std::wstring LineOOShape::GetName() const {
    static const WCHAR name[] = { 0x041B, 0x0456, 0x043D, 0x0456, 0x044F, ' ', 0x0437, ' ', 0x043A, 0x0440, 0x0443,
0x0436, 0x0435, 0x0447, 0x043A, 0x0430, 0x043C, 0x0438, 0 }; // Лінія з кружечками
    return name;
}
void CubeShape::Show(HDC hdc) {
    long origX1 = xs1, origY1 = ys1;
    long origX2 = xs2, origY2 = ys2;

    long dx = (origX2 - origX1) / 3;
    long dy = (origY2 - origY1) / 3;

    long f_left = origX1, f_top = origY1;
    long f_right = origX2 - dx, f_bottom = origY2;
    long b_left = f_left + dx, b_top = f_top - dy;
    long b_right = f_right + dx, b_bottom = f_bottom - dy;

    HBRUSH oldBrush = (HBRUSH)SelectObject(hdc, GetStockObject(NULL_BRUSH));
    Rectangle(hdc, f_left, f_top, f_right, f_bottom);
    Rectangle(hdc, b_left, b_top, b_right, b_bottom);
    SelectObject(hdc, oldBrush);

    LineShape::Set(f_left, f_top, b_left, b_top); LineShape::Show(hdc);
    LineShape::Set(f_right, f_top, b_right, b_top); LineShape::Show(hdc);
    LineShape::Set(f_left, f_bottom, b_left, b_bottom); LineShape::Show(hdc);
    LineShape::Set(f_right, f_bottom, b_right, b_bottom); LineShape::Show(hdc);

    Shape::Set(origX1, origY1, origX2, origY2);
}
std::wstring CubeShape::GetName() const {
    static const WCHAR name[] = { 0x041A, 0x0430, 0x0440, 0x043A, 0x0430, 0x0441, ' ', 0x043A, 0x0443, 0x0431,
0x0430, 0 }; // Каркас куба
    return name;
}
```

my_table.h

```
#pragma once

#include <windows.h>
#include <commctrl.h>
#include <string>
```

```
#include <vector>



#define WM_USER_HIGHLIGHT_SHAPE (WM_USER + 1)
#define WM_USER_DELETE_SHAPE    (WM_USER + 2)



struct TableRowData {
   std::wstring name;
   long x1, y1, x2, y2;
};



class MyTable {
private:
   HWND hDlg;
   HWND hParent;
   HWND hList;


   static MyTable* instance;


   void InitListView();


   static INT_PTR CALLBACK DlgProc(HWND hDlg, UINT message, WPARAM wParam, LPARAM lParam);

public:
   MyTable();
   ~MyTable();


   void Show(HINSTANCE hInstance, HWND parent);


   void Close();


   void UpdateData(const std::vector<TableRowData>& data);


   HWND GetDialogHandle() const { return hDlg; }
};
```

my_table.cpp

```
#include "my_table.h"
#include "Resource.h"

MyTable* MyTable::instance = nullptr;

MyTable::MyTable() : hDlg(NULL), hParent(NULL), hList(NULL) {
   instance = this;
}
```

```
MyTable::~MyTable() {
    Close();
    instance = nullptr;
}

void MyTable::Show(HINSTANCE hInstance, HWND parent) {
    hParent = parent;

    if (!hDlg) {
        hDlg = CreateDialog(hInstance, MAKEINTRESOURCE(IDD_TABLE), hParent, DlgProc);
    }

    if (hDlg) {
        ShowWindow(hDlg, SW_SHOW);
        SetFocus(hDlg);
    }
}

void MyTable::Close() {
    if (hDlg) {
        DestroyWindow(hDlg);
        hDlg = NULL;
    }
}

void MyTable::InitListView() {
    hList = GetDlgItem(hDlg, IDC_LISTVIEW);

    ListView_SetExtendedListViewStyle(hList, LVS_EX_GRIDLINES | LVS_EX_FULLROWSELECT);

    LVCOLUMN lvc;
    lvc.mask = LVCF_TEXT | LVCF_WIDTH | LVCF_FMT;
    lvc.fmt = LVCFMT_LEFT;

    lvc.cx = 120;
    static WCHAR strName[] = { 0x041D, 0x0430, 0x0437, 0x0432, 0x0430, 0 }; // "Назва"
    lvc.pszText = strName;
    ListView_InsertColumn(hList, 0, &lvc);

    lvc.cx = 50;
    lvc.pszText = (LPWSTR)L"x1"; ListView_InsertColumn(hList, 1, &lvc);
    lvc.pszText = (LPWSTR)L"y1"; ListView_InsertColumn(hList, 2, &lvc);
    lvc.pszText = (LPWSTR)L"x2"; ListView_InsertColumn(hList, 3, &lvc);
    lvc.pszText = (LPWSTR)L"y2"; ListView_InsertColumn(hList, 4, &lvc);
}

void MyTable::UpdateData(const std::vector<TableRowData>& data) {
    if (!hList) return;

    ListView_DeleteAllItems(hList);

    LVITEM lvi;
```

```
    lvi.mask = LVIF_TEXT;

    for (size_t i = 0; i < data.size(); i++) {
        lvi.iItem = (int)i;
        lvi.iSubItem = 0;
        lvi.pszText = (LPWSTR)data[i].name.c_str();
        ListView_InsertItem(hList, &lvi);

        std::wstring sx1 = std::to_wstring(data[i].x1);
        std::wstring sy1 = std::to_wstring(data[i].y1);
        std::wstring sx2 = std::to_wstring(data[i].x2);
        std::wstring sy2 = std::to_wstring(data[i].y2);

        ListView_SetItemText(hList, (int)i, 1, (LPWSTR)sx1.c_str());
        ListView_SetItemText(hList, (int)i, 2, (LPWSTR)sy1.c_str());
        ListView_SetItemText(hList, (int)i, 3, (LPWSTR)sx2.c_str());
        ListView_SetItemText(hList, (int)i, 4, (LPWSTR)sy2.c_str());
    }
}

//
INT_PTR CALLBACK MyTable::DlgProc(HWND hDlg, UINT message, WPARAM wParam, LPARAM lParam) {
    switch (message) {

    case WM_INITDIALOG:
        if (instance) {
            instance->hDlg = hDlg;
            instance->InitListView();
        }
        return (INT_PTR)TRUE;

    case WM_NOTIFY: {
        LPNMHDR pnmh = (LPNMHDR)lParam;

        if (pnmh->idFrom == IDC_LISTVIEW) {
            LPNMITEMACTIVATE lpnmitem = (LPNMITEMACTIVATE)lParam;

            if (pnmh->code == NM_CLICK && lpnmitem->iItem != -1) {
                if (instance && instance->hParent) {
                    SendMessage(instance->hParent, WM_USER_HIGHLIGHT_SHAPE, (WPARAM)lpnmitem->iItem, 0);
                }
            }
            else if (pnmh->code == NM_DBLCLK && lpnmitem->iItem != -1) {
                if (instance && instance->hParent) {
                    SendMessage(instance->hParent, WM_USER_DELETE_SHAPE, (WPARAM)lpnmitem->iItem, 0);
                }
            }
        }
        break;
    }

    case WM_COMMAND:
        if (LOWORD(wParam) == IDCANCEL || LOWORD(wParam) == IDOK) {
```

```
            if (instance) instance->Close();
            return (INT_PTR)TRUE;
        }
        break;

    case WM_CLOSE:
        if (instance) instance->Close();
        return (INT_PTR)TRUE;
    }

    return (INT_PTR)FALSE;
}
```

Lab5.cpp

```
#include "framework.h"
#include "Resource.h"
#include "editor.h"
#include "my_table.h"
#include <commdlg.h>

#pragma comment(lib, "comctl32.lib")

HINSTANCE hInst;
WCHAR szTitle[100] = L"Lab5 - Багатовіконний інтерфейс";
WCHAR szWindowClass[100] = L"Lab5Class";

MyTable myTable;

ATOM MyRegisterClass(HINSTANCE hInstance);
BOOL InitInstance(HINSTANCE, int);
LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);



void UpdateTableDisplay() {
    std::vector<TableRowData> data;
    MyEditor& editor = MyEditor::getInstance();

    for (int i = 0; i < editor.GetShapeCount(); i++) {
        Shape* s = editor.GetShape(i);
        if (s) {
            data.push_back({ s->GetName(), s->GetX1(), s->GetY1(), s->GetX2(), s->GetY2() });
        }
    }
    myTable.UpdateData(data);
}

int APIENTRY wWinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE hPrevInstance, _In_ LPWSTR lpCmdLine,
_In_ int nCmdShow)
{
    UNREFERENCED_PARAMETER(hPrevInstance);
    UNREFERENCED_PARAMETER(lpCmdLine);
```

```
    MyRegisterClass(hInstance);
    if (!InitInstance(hInstance, nCmdShow)) return FALSE;

    MSG msg;

    while (GetMessage(&msg, nullptr, 0, 0))
    {
        HWND hDlg = myTable.GetDialogHandle();
        if (hDlg == NULL || !IsDialogMessage(hDlg, &msg)) {
            TranslateMessage(&msg);
            DispatchMessage(&msg);
        }
    }
    return (int)msg.wParam;
}

ATOM MyRegisterClass(HINSTANCE hInstance) {
    WNDCLASSEXW wcex;
    wcex.cbSize = sizeof(WNDCLASSEX);
    wcex.style = CS_HREDRAW | CS_VREDRAW;
    wcex.lpfnWndProc = WndProc;
    wcex.cbClsExtra = 0;
    wcex.cbWndExtra = 0;
    wcex.hInstance = hInstance;
    wcex.hIcon = LoadIcon(nullptr, IDI_APPLICATION);
    wcex.hCursor = LoadCursor(nullptr, IDC_ARROW);
    wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wcex.lpszMenuName = MAKEINTRESOURCEW(IDC_LAB5);
    wcex.lpszClassName = szWindowClass;
    wcex.hIconSm = LoadIcon(nullptr, IDI_APPLICATION);
    return RegisterClassExW(&wcex);
}

BOOL InitInstance(HINSTANCE hInstance, int nCmdShow) {
    hInst = hInstance;
    HWND hWnd = CreateWindowW(szWindowClass, szTitle, WS_OVERLAPPEDWINDOW,
        CW_USEDEFAULT, 0, CW_USEDEFAULT, 0, nullptr, nullptr, hInstance, nullptr);
    if (!hWnd) return FALSE;
    ShowWindow(hWnd, nCmdShow);
    UpdateWindow(hWnd);
    return TRUE;
}

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam) {
    MyEditor& editor = MyEditor::getInstance();

    switch (message) {

    case WM_CREATE:
        editor.Start(hWnd);
        break;
```

```
case WM_LBUTTONDOWN:
    editor.OnLButtonDown(lParam);
    break;

case WM_MOUSEMOVE:
    editor.OnMouseMove(lParam);
    break;

case WM_LBUTTONUP:
    editor.OnLButtonUp(lParam);
    UpdateTableDisplay();
    break;

case WM_PAINT:
    editor.OnPaint();
    break;

case WM_USER_HIGHLIGHT_SHAPE:
    editor.HighlightShape((int)wParam);
    break;

case WM_USER_DELETE_SHAPE:
    editor.DeleteShape((int)wParam);
    UpdateTableDisplay();
    break;

case WM_COMMAND: {
    int wmId = LOWORD(wParam);
    switch (wmId) {

    case IDM_EXIT:
        DestroyWindow(hWnd);
        break;

    case ID_VIEW_TABLE:
        myTable.Show(hInst, hWnd);
        UpdateTableDisplay();
        break;

    case ID_FILE_SAVE_CSV: {
        OPENFILENAMEW ofn;
        WCHAR szFile[260] = { 0 };

        ZeroMemory(&ofn, sizeof(ofn));
        ofn.lStructSize = sizeof(ofn);
        ofn.hwndOwner = hWnd;
        ofn.lpstrFile = szFile;
        ofn.nMaxFile = sizeof(szFile) / sizeof(WCHAR);
        ofn.lpstrFilter = L"CSV Файли (*.csv)\0*.csv\0Всі файли (*.*)\0*.*\0";
        ofn.nFilterIndex = 1;
        ofn.lpstrDefExt = L"csv";
        ofn.Flags = OFN_PATHMUSTEXIST | OFN_OVERWRITEPROMPT;
```

```
            if (GetSaveFileNameW(&ofn) == TRUE) {
                if (editor.SaveToFile(ofn.lpstrFile)) {
                    MessageBoxW(hWnd, L"Дані успішно збережено!", L"Успіх", MB_OK | MB_ICONINFORMATION);
                }
                else {
                    MessageBoxW(hWnd, L"Помилка запису у файл.", L"Помилка", MB_OK | MB_ICONERROR);
                }
            }
            break;
        }


        case ID_FILE_OPEN_CSV: {
            OPENFILENAMEW ofn;
            WCHAR szFile[260] = { 0 };

            ZeroMemory(&ofn, sizeof(ofn));
            ofn.lStructSize = sizeof(ofn);
            ofn.hwndOwner = hWnd;
            ofn.lpstrFile = szFile;
            ofn.nMaxFile = sizeof(szFile) / sizeof(WCHAR);
            ofn.lpstrFilter = L"CSV Файли (*.csv)\0*.csv\0Всі файли (*.*)\0*.*\0";
            ofn.nFilterIndex = 1;
            ofn.Flags = OFN_PATHMUSTEXIST | OFN_FILEMUSTEXIST;

            if (GetOpenFileNameW(&ofn) == TRUE) {
                if (editor.LoadFromFile(ofn.lpstrFile)) {
                    UpdateTableDisplay();

                    MessageBoxW(hWnd, L"Дані успішно завантажено!", L"Успіх", MB_OK | MB_ICONINFORMATION);
                }
                else {
                    MessageBoxW(hWnd, L"Помилка читання або некоректний формат файлу.", L"Помилка", MB_OK |
MB_ICONERROR);
                }
            }
            break;
        }

        default:
            editor.OnCommand(wParam);
            break;
        }
        break;
    }

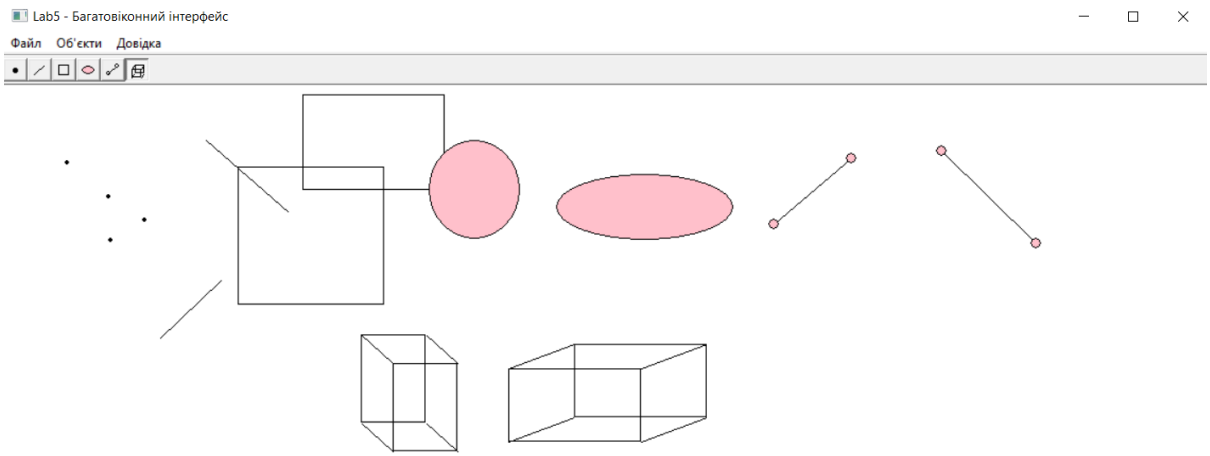
    case WM_INITMENUPOPUP:
        editor.OnInitMenuPopup(wParam, lParam);
        break;

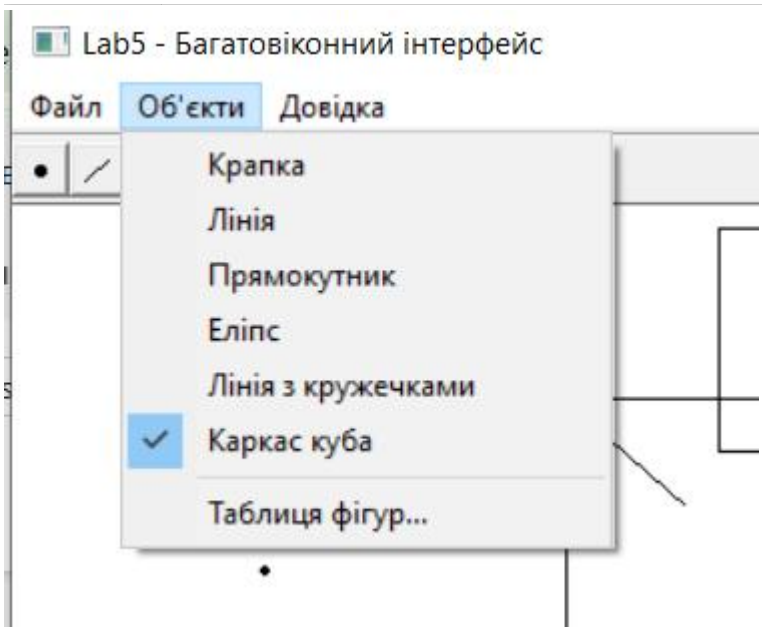
    case WM_NOTIFY:
        return editor.OnNotify(lParam);
```

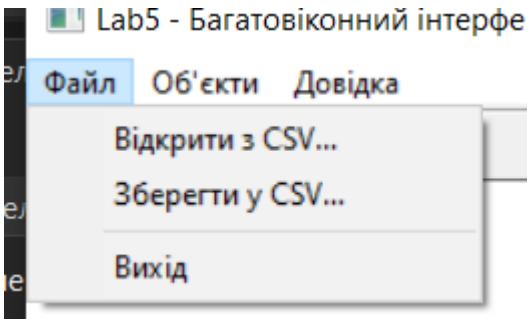
```
    case WM_DESTROY:
      myTable.Close();
      PostQuitMessage(0);
      break;

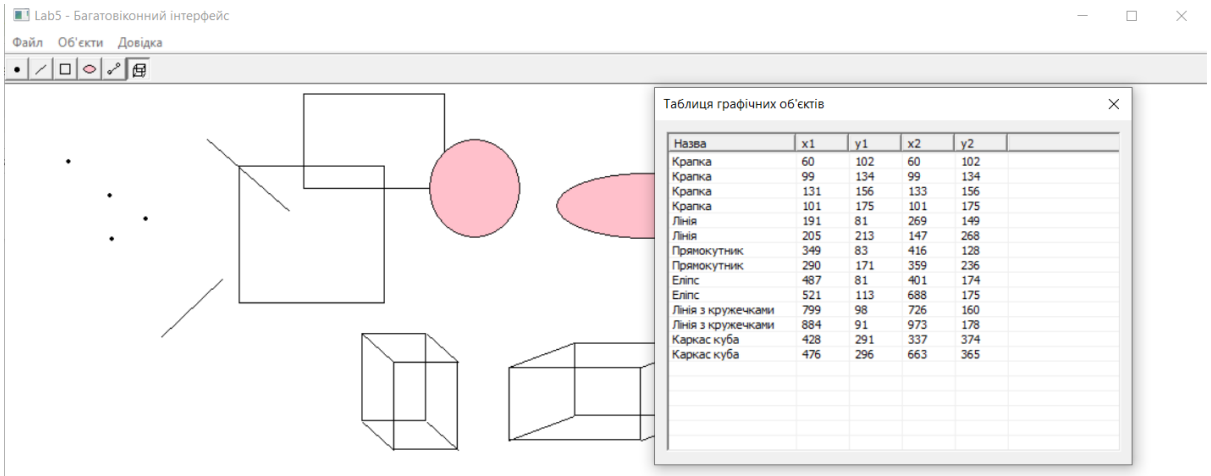
    default:
      return DefWindowProc(hWnd, message, wParam, lParam);
    }
    return 0;
}
```

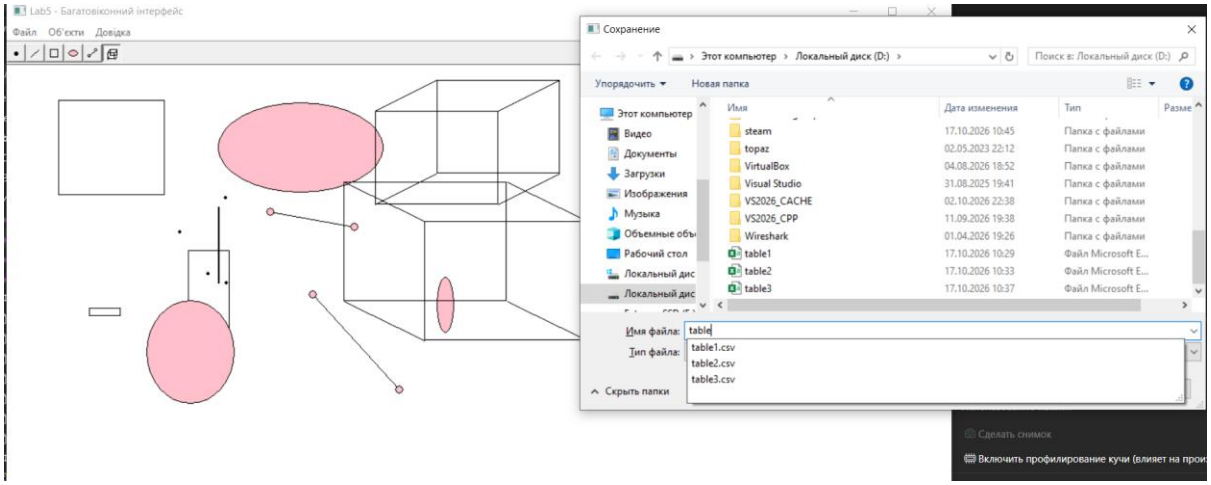
Скріншоти:






Lab5 - Багатовіконний інтерфе
Файл
Об'єкти
Довідка
Відкрити з CSV...
Зберегти у CSV...
Вихід


Lab5 - Багатовіконний інтерфейс
Файл Об'єкти Довідка
Таблиця графічних об'єктів
Назва x1 y1 x2 y2
Крапка 60 102 60 102
Крапка 99 134 99 134
Крапка 131 156 133 156
Крапка 101 175 101 175
Лінія 191 81 269 149
Лінія 205 213 147 268
Прямокутник 349 83 416 128
Прямокутник 290 171 359 236
Еліпс 487 81 401 174
Еліпс 521 113 688 175
Лінія з кружечками 799 98 726 160
Лінія з кружечками 884 91 973 178
Каркас куба 428 291 337 374
Каркас куба 476 296 663 365


Lab5 - Багатовіконний інтерфейс
Файл Об'єкти Довідка
Сохранение
Этот компьютер › Локальный диск (D:) ›
Упорядочить
Новая папка
Этот компьютер
Видео
Документы
Загрузки
Изображения
Музыка
Рабочий стол
Имя
Дата изменения
Тип
steam
topaz
VirtualBox
Visual Studio
VS2026_CACHE
VS2026_CPP
Wireshark
table1
table2
table3
Имя файла: table
Тип файла:
table1.csv
table2.csv
table3.csv
Скрыть папки
Сделать снимок

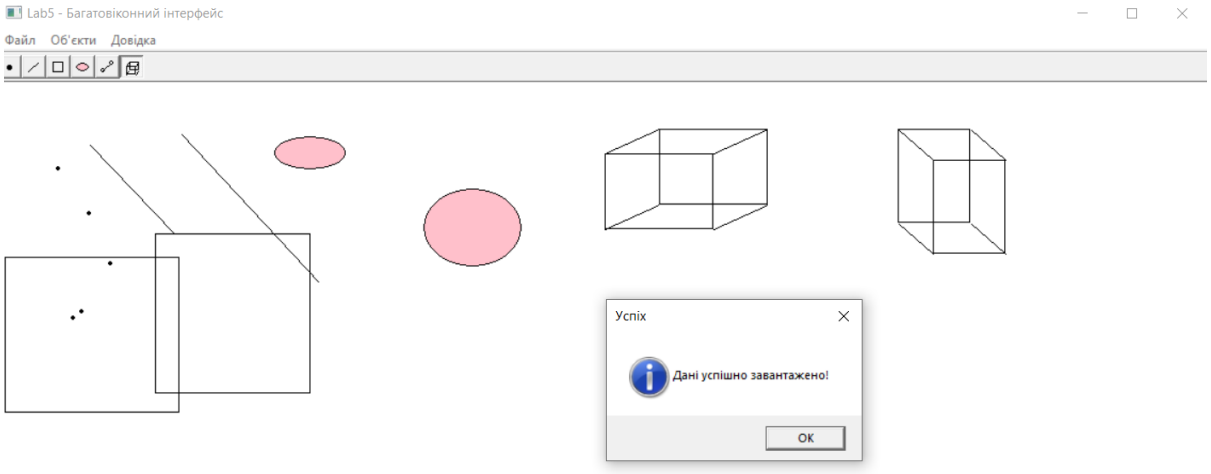
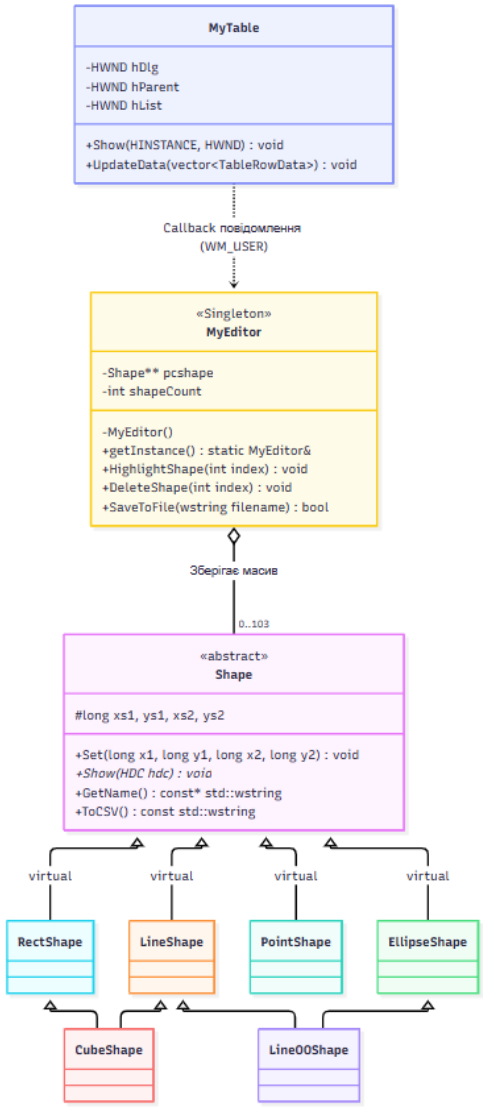

Lab5 - Багатовіконний інтерфейс
Файл Об'єкти Довідка
Успіх
Дані успішно завантажено!
OK

Схема успадкування класів:



Висновок:

Під час виконання лабораторної роботи було успішно розроблено багатовіконний графічний інтерфейс користувача. Я здобув практичні навички впровадження патерну **Singleton Меєрса** для контролю єдиного екземпляра менеджера фігур, а також інтегрував **немодальне діалогове вікно** `MyTable` для відображення властивостей об'єктів. Додатково було засвоєно механізми серіалізації даних у формат CSV та

принципи роботи патерну **Observer/Callback** для ізольованої взаємодії модулів через кастомні повідомлення Windows (`WM_USER`).